QUEENSWOOD SUMMER CAMP

Database Design Documentation

Manuela Planelles Lucas

04.2026 **FP GS DAW – Base de Datos**

1. ER Diagram 3
2. Relational model 3
3. Database Script 4
4. Database Script with data 6
5. Triggers Script 6
6. Procedures Script 10

QUEENSWOOD SUMMER CAMP

Queenswood is a summer camp for teenagers aged between 12 and 15 years old. For each child who enrols in the camp, an identifier, name, gender (M or F), and date of birth must be recorded. The parents' details are also required, including identifier, their names, surnames, genders and phone numbers. It is possible that only one parent's data can be obtained or the two of them, but at least one.

It is possible that some of the kids may have medical issues, so it is necessary to keep a record for each issue and each child. Each of these records is numbered starting from one for each individual child. Not all children have medical issues. For each record, the following information is stored: number, issue description, medication, and special needs.

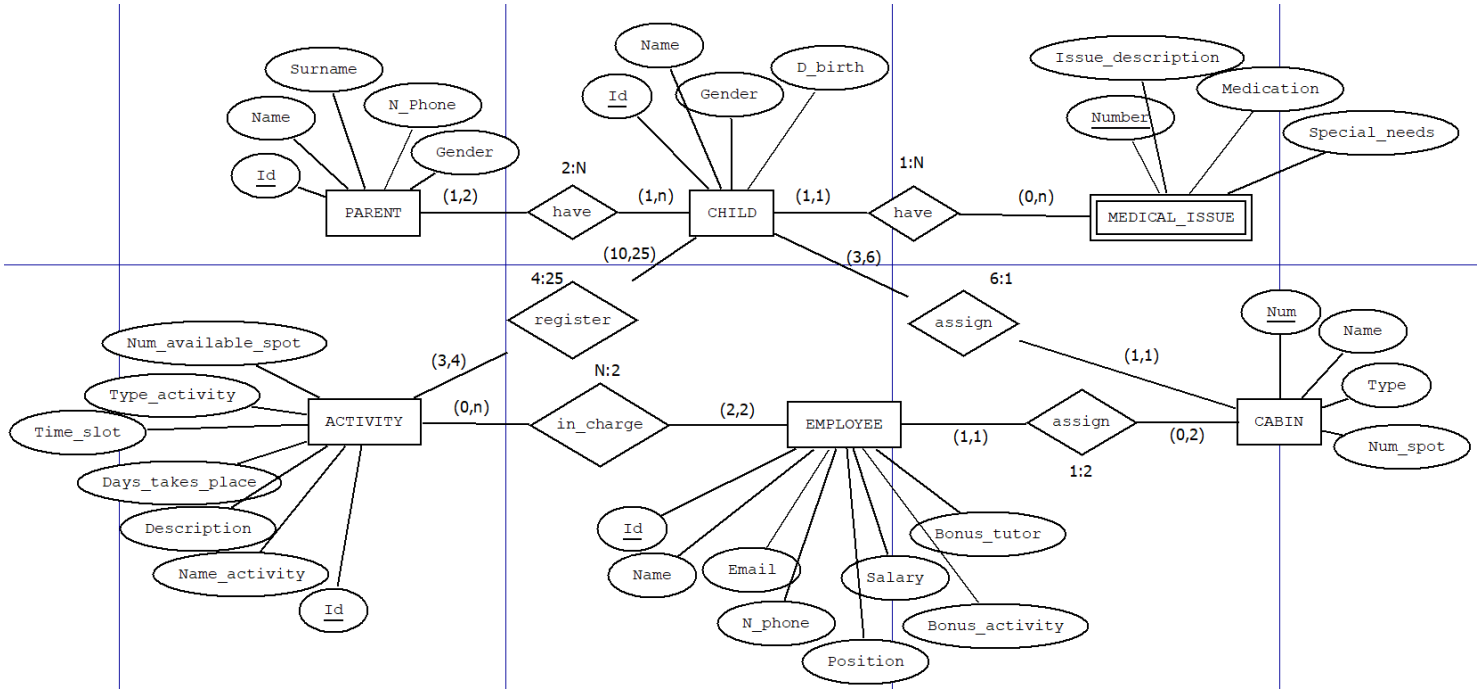
The summer camp offers various physical and recreational activities. Each kid must register for at least three activities and a maximum of four. For each activity, the following information is stored: identifier, name of the activity, description, the days on which it takes place (Mon-Wed-Fri or Tue-Thu-Sat), time slot (10:00–12:00, 16:00–18:00, or 19:00–21:00), type of activity (Physical, Recreational), employees in charge, and the number of available spots. The minimum number of spots for an activity is 10 and the maximum is 25.

A maximum of two activities can take place simultaneously within the same time slot and set of days. For example, Yoga for Teens and Drama Workshop, are simultaneous, Mon-Wed-Fri from 10:00–12:00. Additionally, each activity must have two employees in charge. Note that an employee cannot be assigned to an activity that overlaps in time slot and days with another activity they are already assigned to. It is possible that an employee may not be assigned to any activity at all. Only coach employees may be in charge of activities. Each time an employee is assigned to a specific activity, a 50€ bonus is added to their salary.

Kids are assigned to cabins. Each cabin has a number, name, type (for females or males), and number of spots. Each kid is assigned to only one cabin for the entire holiday period. The minimum number of spots for each cabin is 3 and the maximum is 6. One employee is assigned as the tutor of each cabin.

Each time an employee is assigned to a be the tutor of a specific cabin, a 40€ bonus is added to their salary. Note that there could not be different children gender assigned to the same cabin. For each employee, the following information is stored: identifier, name, email, phone number, position, salary, activities bonus, and tutor bonus. Keep in mind that the bonus are 0 when the data of the employee is recorded.

1. ER Diagram



2. Relational model

PARENT (Id, Name, Surname, N_Phone, Gender)

CHILD (Id, Name, Gender, D_birth, IdParent1 → PARENT, IdParent2 → PARENT, Num→CABIN)

MEDICAL_ISSUE (Number, IdChild → CHILD, Issue_description, Medication, Special_needs)

ACTIVITY (Id, Name_activity, Description, Days_takes_place, Time_slot, Type_activity, Num_available_spot, IdEmployee1 → EMPLOYEE, IdEmployee2 → EMPLOYEE)

EMPLOYEE (Id, Name, Email, N_phone, Position, Salary, Bonus_activity, Bonus_tutor)

CABIN (Num, Name, Type, Num_spot, IdEmployee → EMPLOYEE)

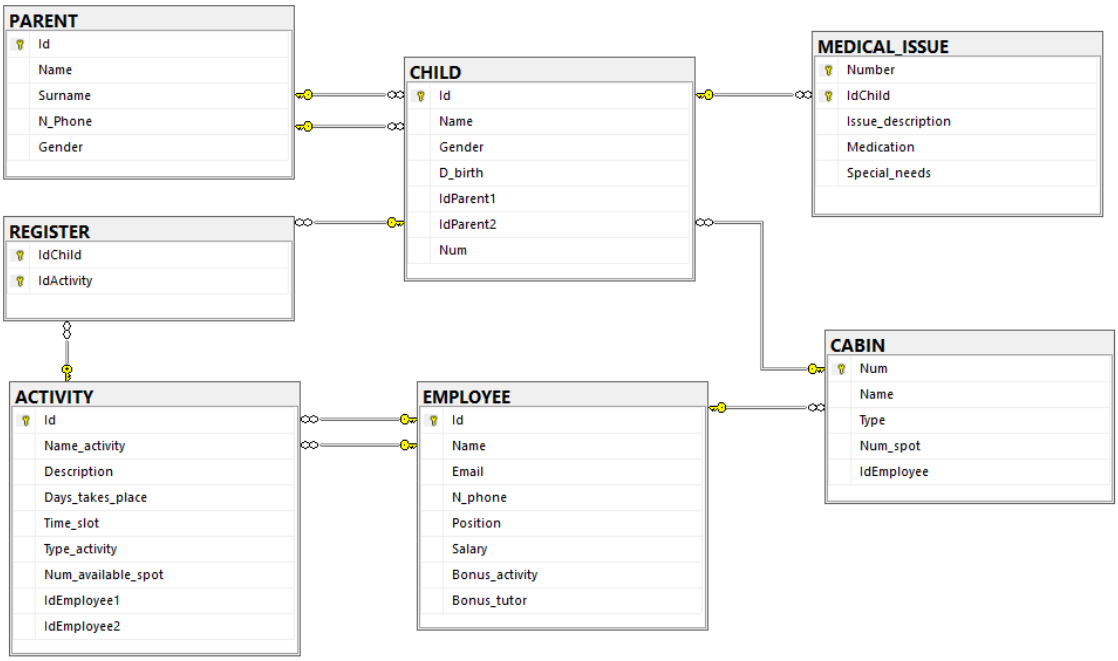
REGISTER (IdChild → CHILD, IdActivity → ACTIVITY)

3. Database Script

```sql
create database QUEENSWOOD_SUMMER_CAMP
use QUEENSWOOD_SUMMER_CAMP

create table PARENT(
Id tinyint identity primary key
, Name varchar (20) not null
, Surname varchar (20) not null
, N_Phone char(9) not null
, Gender char(1) not null
, check (N_Phone like ('[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'))
, check (Gender in ('M','F'))
)
create table EMPLOYEE(
Id tinyint identity primary key
, Name varchar(30) not null
, Email varchar(100) not null unique
, N_phone char (9) not null
, Position varchar(7) not null
, Salary smallmoney not null
, Bonus_activity smallmoney not null default 0
, Bonus_tutor smallmoney not null default 0
, check (Email like ('_%@_%._%'))
, check (N_phone like ('[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'))
, check (Position in ('Manager','Coach','Cook'))
, check (Salary > 0)
)
create table CABIN(
Num tinyint identity primary key
, Name varchar(10) not null
, Type char(1) not null
, Num_spot tinyint not null
, IdEmployee tinyint
, foreign key (IdEmployee) references EMPLOYEE (Id)
, check (Type in ('F','M'))
, check (Num_spot >= 3 and Num_spot <= 6)
)
create table CHILD(
Id tinyint identity primary key
, Name varchar (30) not null
, Gender char(1) not null
, D_birth date not null
, IdParent1 tinyint not null
, IdParent2 tinyint
, Num tinyint
, foreign key (IdParent1) references PARENT (Id)
, foreign key (IdParent2) references PARENT (Id)
, foreign key (Num) references CABIN (Num)
, check (Gender in ('M','F'))
)
```

```sql
create table MEDICAL_ISSUE(
Number tinyint not null
, IdChild tinyint not null
, Issue_description varchar(50) not null
, Medication varchar(30) not null
, Special_needs varchar(40) not null
, primary key (Number,IdChild)
, foreign key (IdChild) references CHILD (Id)
)
create table ACTIVITY(
Id tinyint identity primary key
, Name_activity varchar(25) not null
, Description varchar(60) not null
, Days_takes_place varchar(11) not null
, Time_slot varchar(11) not null
, Type_activity varchar(12) not null
, Num_available_spot tinyint not null
, IdEmployee1 tinyint not null
, IdEmployee2 tinyint not null
, foreign key (IdEmployee1) references EMPLOYEE (Id)
, foreign key (IdEmployee2) references EMPLOYEE (Id)
, check (Days_takes_place in ('Mon-Wed-Fri','Tue-Thu-Sat'))
, check (Time_slot in ('10:00-12:00','16:00-18:00','19:00-21:00'))
, check (Type_activity in ('Physical','Recreational'))
, check (Num_available_spot >= 10 and Num_available_spot <= 25)
)
create table REGISTER(
IdChild tinyint not null
, IdActivity tinyint not null
, primary key (IdChild, IdActivity)
, foreign key (IdChild) references CHILD (Id)
, foreign key (IdActivity) references ACTIVITY (Id)
)
```



4. Database Script with data

En este apartado se ha realizado la inserción de datos en todas las tablas de la base de datos, siguiendo el orden de dependencias entre tablas para evitar errores de foreign key. Se han insertado 11 empleados, 30 padres, 9 cabañas, 40 niños, 6 registros médicos, 9 actividades y las inscripciones de los niños en las actividades. Los campos que serán gestionados automáticamente por los triggers (asignación de cabañas, coaches y bonuses) se han dejado sin asignar, ya que serán rellenados al ejecutar los triggers correspondientes. Se adjunta el archivo SQL completo.

[Insert.sql]

5. Triggers Script

ASSIGN_CABIN. Create a trigger that automatically assigns a cabin to a child when their data is **inserted**. Take into account the child's gender and ensure that the number of available spots for each cabin is not exceeded.

```
create or alter trigger ASSIGN_CABIN
on CHILD
instead of INSERT
as begin
    declare @name varchar(30) = (select Name from inserted)
    declare @gender char(1) = (select Gender from inserted)
    declare @dbirth date = (select D_birth from inserted)
    declare @parent1 tinyint = (select IdParent1 from inserted)
    declare @parent2 tinyint = (select IdParent2 from inserted)

    declare @cabin tinyint = (select top 1 Num
                              from CABIN
                              where Type = @gender
                                   and Num_spot > (select count(*) from CHILD
where Num = CABIN.Num))

    insert into CHILD (Name, Gender, D_birth, IdParent1, IdParent2, Num)
    values (@name, @gender, @dbirth, @parent1, @parent2, @cabin)
end
```

UPDATE_EMPLOYEE_ACTIVITY. Create a trigger to randomly assign the employees who will be in charge when a new activity is **inserted**. Take into account the following:

- An employee cannot be in charge of more than one activity that takes place during the same time slot and on the same days as another activity they have already been assigned to.
- Each activity must have two different employees in charge.
- A 50€ bonus will be added to each employee's salary for every activity they are assigned to.
- Check the coaches that do not have activities or are not in charge of a lot of activities and assigned them to be in charge of the cabins (update).

```
create or alter trigger UPDATE_EMPLOYEE_ACTIVITY
on ACTIVITY
instead of INSERT
as begin
    declare @name varchar(25) = (select Name_activity from inserted)
    declare @description varchar(60) = (select Description from inserted)
    declare @days varchar(11) = (select Days_takes_place from inserted)
    declare @slot varchar(11) = (select Time_slot from inserted)
    declare @type varchar(12) = (select Type_activity from inserted)
    declare @spots tinyint = (select Num_available_spot from inserted)

    declare @coach1 tinyint = (select top 1 Id from EMPLOYEE
                               where Position = 'Coach'
                               and Id not in (select IdEmployee1
                                              from ACTIVITY
                                              where Days_takes_place = @days
                                              and Time_slot = @slot
                                              and IdEmployee1 is not null)
                               and Id not in (select IdEmployee2
                                              from ACTIVITY
                                              where Days_takes_place = @days
                                                    and Time_slot = @slot
                                                    and IdEmployee2 is not
                                                    null)
                               order by NEWID())

    declare @coach2 tinyint = (select top 1 Id
                              from EMPLOYEE
                              where Position = 'Coach'
                               and Id <> @coach1
                               and Id not in (select IdEmployee1
                                              from ACTIVITY
                                              where Days_takes_place = @days
                                                    and Time_slot = @slot
                                                    and IdEmployee1 is not
                                                    null)
                               and Id not in (select IdEmployee2
                                              from ACTIVITY
                                              where Days_takes_place = @days
                                                    and Time_slot = @slot
```

```
                                                        and IdEmployee2 is not
                                                        null)
                                    order by NEWID())

    insert into ACTIVITY (Name_activity, Description, Days_takes_place,
Time_slot, Type_activity, Num_available_spot, IdEmployee1, IdEmployee2)
    values (@name, @description, @days, @slot, @type, @spots, @coach1,
@coach2)

   update EMPLOYEE
   set Bonus_activity = Bonus_activity + 50 where Id = @coach1
   update EMPLOYEE
   set Bonus_activity = Bonus_activity + 50 where Id = @coach2
end
```

UPDATE_BONUS_TUTOR. Write a trigger to update the salary with the tutor bonus every time that a tutor is assigned to the tutor of a cabin (when the data of the cabin is **updated**). Keep in mind that a tutor can lead several cabins.

```
create or alter trigger UPDATE_BONUS_TUTOR
on CABIN
after UPDATE
as begin
   declare @tutor_new tinyint = (select IdEmployee from inserted)
    declare @tutor_old tinyint = (select IdEmployee from deleted)

    if @tutor_old is null or @tutor_old <> @tutor_new
    begin
        update EMPLOYEE
        set Bonus_tutor = Bonus_tutor + 40
        where Id = @tutor_new
    end
end
```

APPLY_ACTIVITY. Create a trigger that is fired each time a child signs up for an activity (**insert**). Take into account the following:

- Check the number of available spots for the activity. If there are not enough spots, print a message and roll back the insertion. 'There are not places enough'
- Verify that the child is not already enrolled in another activity that takes place during the same time slot and on the same days. If this is the case, print a message and roll back the insertion. Name 'already applied for an activity at the same time and days'
- Ensure that the child is not already enrolled in 4 activities. If this limit has been reached, print a message and roll back the insertion. Name 'already applied for 4 activities'

```
create or alter trigger APPLY_ACTIVITY
on REGISTER
after INSERT
```

```
as begin
    declare @child tinyint = (select IdChild from inserted)
    declare @activity tinyint = (select IdActivity from inserted)

    declare @inscritos int = (select count(*) from REGISTER where IdActivity
= @activity)
    declare @plazas int = (select Num_available_spot from ACTIVITY where Id =
@activity)

    declare @days varchar(11) = (select Days_takes_place from ACTIVITY where
Id = @activity)
    declare @slot varchar(11) = (select Time_slot from ACTIVITY where Id =
@activity)

    declare @inscritos_child int = (select count(*) from REGISTER where
IdChild = @child)
    declare @name varchar(30) = (select Name from CHILD where Id = @child)

    --Comprueba el número de plazas disponibles para la actividad
    if @inscritos > @plazas
        begin
            print 'There are not places enough'
            rollback transaction
            return
        end

    --Verifica que el niño no esté ya inscrito en otra actividad que tenga
lugar durante el mismo horario
    if exists (select * from REGISTER, ACTIVITY
                where REGISTER.IdActivity = ACTIVITY.Id
                  and REGISTER.IdChild = @child
                  and Days_takes_place = @days
                  and Time_slot = @slot
                  and ACTIVITY.Id <> @activity)
        begin
             print @name + ' already applied for an activity at the same time
and days'
            rollback transaction
            return
        end

    --Asegúrate de que el niño no esté ya inscrito en 4 actividades
    if @inscritos_child >4
        begin
            print @name + ' already applied for 4 activities'
            rollback transaction
            return
        end
end
```

6. Procedures Script

LIST_MEDICAL_RECORDS. Create a procedure to display all the medical records for all the kids.

```
create or alter procedure LIST_MEDICAL_RECORDS
as begin
    declare @idChild tinyint
    declare @name varchar(30), @gender varchar(6), @age int
    declare @list varchar(max) = ''

    declare cursor_medical cursor
    for select distinct IdChild, Name
        from MEDICAL_ISSUE, CHILD
        where MEDICAL_ISSUE.IdChild = CHILD.Id
        order by Name

    open cursor_medical
    fetch next from cursor_medical into @idChild, @name

    while @@FETCH_STATUS = 0
    begin
        set @name = (select Name from CHILD where Id = @idChild)
        set @gender = (select case when Gender = 'F' then 'Female' else
'Male' end from CHILD where Id = @idChild)
        set @age = (select datediff(year, D_birth, getdate()) from CHILD
where Id = @idChild)

        print 'Name: ' + @name + space(5) + @gender + space(5) + 'Age: ' +
cast(@age as varchar)
        print replicate('-', 50)

        select @list = @list +
            space(4) + cast(Number as char(4)) +
            Issue_description + char(10) +
            space(8) + 'Medication: ' + Medication + char(10) +
            space(8) + 'Special Needs: ' + Special_needs + char(10)+
            char(10)
        from MEDICAL_ISSUE
        where IdChild = @idChild

        print @list
        set @list = ''

        fetch next from cursor_medical into @idChild, @name
    end

    close cursor_medical
    deallocate cursor_medical
end
```

ACTIVITIES_TYPE. Create a procedure to display the activities and the kids that applied for them for the type of activity given as parameter.

```
create or alter procedure ACTIVITIES_TYPE (@type varchar(12))
as begin
    declare @idActivity tinyint
    declare @nameActivity varchar(25), @description varchar(60), @days
varchar(11), @time varchar(12), @spot tinyint
    declare @coach1 varchar(30), @coach2 varchar(30)
    declare @list varchar(max) = ''

    declare cursor_type cursor
    for select Id from ACTIVITY where Type_activity = @type

    open cursor_type
    fetch next from cursor_type into @idActivity

    while @@FETCH_STATUS = 0
    begin
        set @nameActivity = (select Name_activity from ACTIVITY where Id =
@idActivity)
        set @description = (select Description from ACTIVITY where Id =
@idActivity)
        set @days = (select Days_takes_place from ACTIVITY where Id =
@idActivity)
        set @time = (select Time_slot from ACTIVITY where Id = @idActivity)
        set @spot = (select Num_available_spot from ACTIVITY where Id =
@idActivity)
        set @coach1 = (select Name from EMPLOYEE where Id = (select
IdEmployee1 from ACTIVITY where Id = @idActivity))
        set @coach2 = (select Name from EMPLOYEE where Id = (select
IdEmployee2 from ACTIVITY where Id = @idActivity))

        print replicate('-', 50)
        print @nameActivity
        print @description
        print @days + space(3) + @time + space(5) + 'Available Spots: ' +
cast(@spot as varchar)
        print 'Coach1: ' + @coach1 + space(10) + 'Coach2: ' + @coach2
        print replicate('-', 50)
        print 'CHILD' + space(13) + 'GENDER' + space(2) + 'AGE' + space(3) +
'PARENT1' + space(13) + 'PARENT2'

        select @list = @list +
            cast(CHILD.Name as char(20)) +
            cast(CHILD.Gender as char(5)) +
            cast(datediff(year, CHILD.D_birth, getdate()) as char(5)) +
            cast(concat(P1.Name, ' ', P1.Surname) as char(20)) +
            case when CHILD.IdParent2 is null then '---'
                 else cast(concat(P2.Name, ' ', P2.Surname) as char(20))
            end + char(10)
        from REGISTER
            join CHILD on REGISTER.IdChild = CHILD.Id
            join PARENT P1 on CHILD.IdParent1 = P1.Id
```

```
            left join PARENT P2 on CHILD.IdParent2 = P2.Id
        where REGISTER.IdActivity = @idActivity

        print @list
        set @list = ''
        fetch next from cursor_type into @idActivity
    end

    close cursor_type
    deallocate cursor_type
end
```

CABIN_CHILDREN. Create a procedure to display all cabins of the type given as a parameter and the children who are assigned to each of them. Display as well all the activities the kids applied for.

```
create or alter procedure CABIN_CHILDREN (@type char(1))
as begin
    declare @numCabin tinyint
    declare @nameCabin varchar(10), @tutor varchar(30), @slots tinyint
    declare @idChild tinyint, @nameChild varchar(30), @dbirth date

    declare @list varchar(max) = ''

    declare cursor_cabin cursor

    for select CABIN.Num, CHILD.Id
        from CHILD, CABIN
        where CHILD.Num = CABIN.Num
            and CABIN.Type = @type

    open cursor_cabin
    fetch next from cursor_cabin into @numCabin, @idChild

    while @@FETCH_STATUS = 0
        begin
            set @nameCabin = (select Name from CABIN where Num = @numCabin)
            set @tutor = (select Name from EMPLOYEE where Id = (select
IdEmployee from CABIN where Num = @numCabin))
            set @slots = (select Num_spot from CABIN where Num = @numCabin)
            set @nameChild = (select Name from CHILD where Id = @idChild)
            set @dbirth = (select D_birth from CHILD where Id = @idChild)

            print 'Name: ' + @nameCabin + space(5)+ 'Tutor: ' + @tutor +
space(5) + 'N. Slots: ' + cast(@slots as varchar)
            print replicate('-',52)

            print @nameChild + ' - ' + cast(@dbirth as varchar(10))

            select @list = @list +
                space(20) + Name_activity + char(10)
            from REGISTER, ACTIVITY
            where REGISTER.IdActivity = ACTIVITY.Id
                and REGISTER.IdChild = @idChild
```

```
            print @list
            set @list = ''

            fetch next from cursor_cabin into @numCabin, @idChild
        end
    close cursor_cabin
    deallocate cursor_cabin
end
-- exec CABIN_CHILDREN 'F'
-- exec CABIN_CHILDREN 'M'
```

EMPLOYEES_SALARY. Create a procedure to display the data for all the employees.

```
create or alter procedure EMPLOYEES_SALARY
as begin
    declare @list varchar(max) = ''

    print 'Name' + space(15) + 'Phone N.' + space(5) + 'Position' + space(5)
+ 'Salary' + space(5) + 'Bonus Act' + space(5) + 'Bonus Tutor'
    print replicate('-', 85)

    select @list = @list +
        cast(Name as char(19)) +
        cast(N_phone as char(13)) +
        cast(Position as char(10)) +
        cast(cast(Salary as money) as char(10)) +
        cast(cast(Bonus_activity as money) as char(13)) +
        cast(cast(Bonus_tutor as money) as char(16)) + char(10)
    from EMPLOYEE

    print @list
end
```